

LES GÉNÉRIQUES

Les génériques sont une fonctionnalité du framework .NET apparus avec la version 2. Vous vous en souvenez peut-être, nous avons cité le mot dans le chapitre sur les tableaux et les listes. Ils permettent de créer des méthodes ou des classes qui sont indépendantes d'un type. Il est très important de connaître leur fonctionnement car c'est un mécanisme clé qui permet de faire beaucoup de choses, notamment en termes de réutilisabilité et d'amélioration des performances.

Qu'est-ce que les génériques ?

Avec les génériques, vous pouvez **définir des méthodes ou des classes qui sont indépendantes d'un type**. On les appellera des **méthodes génériques** et des **types génériques**.

Nous en avons déjà utilisé, rappelez-vous. La liste est une classe comme nous en avons déjà vu plein, sauf qu'elle a la capacité d'être utilisée avec n'importe quel autre type, comme les entiers, les chaînes de caractères, les voitures ...

Cela permet d'éviter de devoir créer une classe `ListeInt`, une classe `ListeString`, une classe `ListeVoiture`, etc ... On pourra utiliser cette classe avec tous les types grâce aux chevrons :

```
1List<string> listeChaine = new List<string>();
2List<int> listeEntier = new List<int>();
3List<Voiture> listeVoiture = new List<Voiture>();
```

Nous indiquons entre les chevrons le type qui sera utilisé avec le type générique.

Oui mais, si nous voulons pouvoir mettre n'importe quel type d'objet dans une liste, il suffirait de créer une `ListeObject` ? Puisque tous les objets dérivent d'`object`...

En fait, c'est le choix qui avait été fait dans la toute première version de .NET. On utilisait l'objet `ArrayList` qui possède une méthode `Add` prenant en paramètre un `object`. Cela fonctionne. Sauf que nous nous trouvions faces à des limitations :

- Premièrement, nous pouvions mélanger n'importe quel type d'objet dans la liste, des entiers, des voitures, des chiens, etc. Cela devenait une classe fourre-tout et nous ne savions jamais ce qu'il y avait dans la liste.
- Deuxièmement, même si nous savions qu'il n'y avait que des entiers dans la liste, nous étions obligés de le traiter en tant qu'`object` et donc d'utiliser **le boxing** et **l'unboxing** pour mettre les objets dans la liste ou pour les récupérer.

Cela engendrait donc confusion et perte de performance. Grâce aux génériques, il devient donc possible de créer des listes de n'importe quel type en étant certain du type que nous allons récupérer dans la liste.

Les types génériques du framework .NET

Le framework .NET possède beaucoup de classes et d'interfaces génériques, notamment dans l'espace de nom `System.Collections.Generic`.

La liste est la classe générique que vous utiliserez sûrement le plus. Mais beaucoup d'autres sont à votre disposition. Citons par exemple la classe `Queue<>` qui permet de gérer une file d'attente style FIFO (*first in, first out* : premier entré, premier sorti) :

```

1Queue<int> file = new Queue<int>();
2file.Enqueue(3);
3file.Enqueue(1);
4int valeur = file.Dequeue(); // valeur contient 3
5valeur = file.Dequeue(); // valeur contient 1

```

Citons encore le dictionnaire d'élément qui est une espèce d'annuaire où l'on accède aux éléments grâce à une clé :

```

1Dictionary<string, Personne> annuaire = new Dictionary<string, Personne>();
2annuaire.Add("06 01 02 03 04", new Personne { Prenom = "Nicolas"});
3annuaire.Add("06 06 06 06 06", new Personne { Prenom = "Jeremie" });
4
5Personne p = annuaire["06 06 06 06 06"]; // p contient la propriété Prenom valant Jeremie

```

Loin de moi l'idée de vous énumérer toutes les collections génériques du framework .NET, le but est de vous montrer rapidement qu'il existe beaucoup de classes génériques dans le framework .NET.

Créer une méthode générique

Nous commençons à cerner l'intérêt des génériques. Sachez qu'il est bien sûr possible de créer vos propres classes génériques ou vos propres méthodes.

Commençons par les méthodes.

Il est globalement intéressant d'utiliser un type générique partout où nous pourrions avoir un `object`. Soit la classe `Afficheur()` qui prend un objet en paramètre :

```

1public static class Afficheur
2{
3    public static void Affiche(object o)
4    {
5        Console.WriteLine("Afficheur d'objet :");
6        Console.WriteLine("\tType : " + o.GetType());
7        Console.WriteLine("\tReprésentation : " + o.ToString());
8    }
9}

```

Nous avons ici utilisé une classe statique permettant d'afficher le type d'un objet et sa représentation. Nous pouvons l'utiliser ainsi :

```

1int i = 5;
2double d = 9.5;
3string s = "abcd";
4Voiture v = new Voiture();
5
6Afficheur.Affiche(i);
7Afficheur.Affiche(d);
8Afficheur.Affiche(s);
9Afficheur.Affiche(v);

```

Rappelez-vous, à chaque fois qu'on passe dans cette méthode, l'objet est boxé en type `object` quand il s'agit d'un type valeur.

Nous pouvons améliorer cette méthode en créant une méthode générique. Regardons ce code :

```
1public static class Afficheur
2{
3    public static void Affiche<T>(T a)
4    {
5        Console.WriteLine("Afficheur d'objet :");
6        Console.WriteLine("\tType : " + a.GetType());
7        Console.WriteLine("\tReprésentation : " + a.ToString());
8    }
9}
```

Cette méthode fait exactement la même chose mais avec les génériques.

Dans un premier temps, la méthode annonce qu'elle va utiliser un type générique représenté par la lettre T entre chevrons.

Il est conventionnel que les types génériques soient utilisés avec T.

Cela veut dire que tout type utilisé dans cette méthode déclaré avec T sera du type passé à la méthode.

Ainsi, la variable a est du type générique qui sera précisé lors de l'appel à cette méthode.

Comme a est un objet, nous pouvons appeler la méthode GetType() et la méthode ToString() sur cet objet.

Pour afficher un objet, nous pourrions faire :

```
1int i = 5;
2double d = 9.5;
3string s = "abcd";
4Voiture v = new Voiture();
5
6Afficheur.Affiche<int>(i);
7Afficheur.Affiche<double>(d);
8Afficheur.Affiche<string>(s);
9Afficheur.Affiche<Voiture>(v);
```

Dans le premier appel, nous indiquons que nous souhaitons afficher i dont le type générique est int.

C'est comme si le CLR créait la surcharge de la méthode Affiche, prenant un entier en paramètre :

```
1public static void Affiche(int a)
2{
3    Console.WriteLine("Afficheur d'objet :");
4    Console.WriteLine("\tType : " + a.GetType());
5    Console.WriteLine("\tReprésentation : " + a.ToString());
6}
```

De même pour l'affichage suivant, où l'on indique le type double entre les chevrons. C'est comme si le CLR créait la surcharge prenant un double en paramètre :

```
1public static void Affiche(double a)
2{
3    Console.WriteLine("Afficheur d'objet :");
4    Console.WriteLine("\tType : " + a.GetType());
5    Console.WriteLine("\tReprésentation : " + a.ToString());
6}
```

Et ceci pour tous les types utilisés, à savoir ici int, double, string et Voiture.

À noter que dans cet exemple, nous pouvons remplacer les quatre lignes suivantes :

```
1Afficheur.Affiche<int>(i);
2Afficheur.Affiche<double>(d);
3Afficheur.Affiche<string>(s);
4Afficheur.Affiche<Voiture>(v);
```

Par :

```
1Afficheur.Affiche(i);
2Afficheur.Affiche(d);
3Afficheur.Affiche(s);
4Afficheur.Affiche(v);
```

En effet, nul besoin de préciser quel type nous souhaitons traiter ici, le compilateur est assez malin pour le déduire du type de la variable. La variable `i` étant un entier, il est obligatoire que le type générique soit un entier. Il est donc facultatif ici de le préciser.

Une fois qu'il est précisé entre les chevrons, le type générique s'utilise dans la méthode comme n'importe quel autre type. Nous pouvons avoir autant de paramètres génériques que nous le voulons dans les paramètres et utiliser le type générique dans le corps de la méthode.

Par exemple la méthode suivante permet d'échanger le contenu de deux variables entre elles. C'est donc une méthode générique puisqu'elle précise entre les chevrons que nous pourrions utiliser le type `T`.

```
1public static void Echanger<T>(ref T t1, ref T t2)
2{
3    T temp = t1;
4    t1 = t2;
5    t2 = temp;
6}
```

En paramètres de la méthode, nous passons deux variables de types génériques.

Dans le corps de la méthode, nous créons une variable du type générique qui sert de mémoire temporaire puis nous échangeons les références des deux variables.

Nous pourrions utiliser cette méthode ainsi :

```
1int i = 5;
2int j = 10;
3Echanger(ref i, ref j);
4Console.WriteLine(i);
5Console.WriteLine(j);
6
7Voiture v1 = new Voiture { Couleur = "Rouge" };
8Voiture v2 = new Voiture { Couleur = "Verte" };
9Echanger(ref v1, ref v2);
10Console.WriteLine(v1.Couleur);
11Console.WriteLine(v2.Couleur);
```

Il est bien sûr possible de créer des méthodes prenant en paramètres plusieurs types génériques différents. Il suffit alors de préciser autant de types différents entre les chevrons qu'il y a de types génériques différents :

```
1static void Main(string[] args)
2{
3    int i = 5;
4    int j = 5;
5    double d = 9.5;
6
7    Console.WriteLine(EstEgal(i, j));
8    Console.WriteLine(EstEgal(i, d));
9}
10public static bool EstEgal<T, U>(T t, U u)
11{
12    return t.Equals(u);
13}
```

Ici, la méthode `EstEgal()` prend en paramètres potentiellement deux types différents. Nous l'appelons une première fois avec deux entiers et ensuite avec un entier et un double.

Créer une classe générique

Une classe générique fonctionne comme pour les méthodes. C'est une classe où nous pouvons indiquer de 1 à N types génériques. C'est comme cela que fonctionne la liste que nous avons déjà beaucoup manipulée.

En fait, la liste n'est qu'une espèce de tableau évolué. Nous pourrions très bien imaginer créer notre propre liste sur ce principe, sachant que c'est complètement absurde car elle sera forcément moins bien que cette classe, mais c'est pour l'étude.

Le principe est d'avoir un tableau plus ou moins dynamique qui grossit si jamais le nombre d'éléments devient trop grand pour sa capacité. Pour déclarer une classe générique, nous utiliserons à nouveau les chevrons à la fin de la ligne qui déclare la classe :

```
1public class MaListeGenerique<T>
2{
3}
```

Nous allons réaliser une implémentation toute basique de cette classe histoire de voir un peu à quoi ressemble une classe générique. Cette classe n'a d'intérêt que pour étudier les génériques, vous lui préférerez évidemment la classe `List<>` du framework .NET.

Nous avons besoin de trois variables privées. La capacité de la liste, le nombre d'éléments dans la liste et le tableau générique.

```
1public class MaListeGenerique<T>
2{
3    private int capacite;
4    private int nbElements;
5    private T[] tableau;
6
7    public MaListeGenerique()
8    {
9        capacite = 10;
10       nbElements = 0;
11       tableau = new T[capacite];
12    }
13}
```

Notons la déclaration du tableau. Il utilise le type générique. Concrètement, cela veut dire que quand nous utiliserons la liste avec un entier, nous aurons un tableau d'entiers. Lorsque nous utiliserons la liste avec un objet `Voiture`, nous aurons un tableau de `Voiture`, etc.

Nous initialisons ces variables membres dans le constructeur, en décidant complètement arbitrairement que la capacité par défaut de notre liste est de 10 éléments. La dernière ligne instancie le tableau en lui indiquant sa taille.

Il reste à implémenter l'ajout dans la liste :

```
1public class MaListeGenerique<T>
2{
3    [Code enlevé pour plus de clarté]
4
5    public void Ajouter(T element)
6    {
7        if (nbElements >= capacite)
8        {
9            capacite *= 2;
10           T[] copieTableau = new T[capacite];
11           for (int i = 0; i < tableau.Length; i++)
12           {
```

```

13         copieTableau[i] = tableau[i];
14     }
15     tableau = copieTableau;
16 }
17 tableau[nbElements] = element;
18 nbElements++;
19 }
20}

```

Il s'agit simplement de mettre la valeur que l'on souhaite ajouter à l'emplacement adéquat dans le tableau. Nous le mettons en dernière position, c'est-à-dire à l'emplacement correspondant au nombre d'éléments.

Au début, nous avons commencé par vérifier si le nombre d'éléments était supérieur à la capacité du tableau. Si c'est le cas, alors nous devons augmenter la capacité du tableau, j'ai ici décidé encore complètement arbitrairement que je doublais la capacité. Il ne reste plus qu'à créer un nouveau tableau de cette nouvelle capacité et à copier les éléments du premier tableau dans celui-ci.

Vous aurez noté que le paramètre de la méthode `Ajouter` est bien du type générique.

Pour le plaisir, rajoutons enfin une méthode permettant de récupérer l'élément à un indice donné :

```

1public class MaListeGenerique<T>
2{
3    [Code enlevé pour plus de clarté]
4
5    public T ObtenirElement(int indice)
6    {
7        return tableau[indice];
8    }
9}

```

Il s'agit juste d'accéder au tableau pour renvoyer la valeur à l'indice concerné. L'élément intéressant ici est de constater que le type de retour de la méthode est bien du type générique.

Cette liste peut s'utiliser de la manière suivante :

```

1MaListeGenerique<int> maListe = new MaListeGenerique<int>();
2maListe.Ajouter(25);
3maListe.Ajouter(30);
4maListe.Ajouter(5);
5
6Console.WriteLine(maListe.ObtenirElement(0));
7Console.WriteLine(maListe.ObtenirElement(1));
8Console.WriteLine(maListe.ObtenirElement(2));
9
10for (int i = 0; i < 30; i++)
11{
12    maListe.Ajouter(i);
13}

```

Ici, nous utilisons la liste avec un entier, mais elle fonctionnerait tout aussi bien avec un autre type. N'hésitez pas à passer en debug dans la méthode `Ajouter()` pour observer ce qu'il se passe exactement lors de l'augmentation de capacité.

Voilà comment on crée une classe générique !

Une fois qu'on a compris que le type générique s'utilise comme n'importe quel autre type, cela devient assez facile.

Rappelez-vous, toute classe qui manipule des `object` est susceptible d'être améliorée en utilisant les génériques.

La valeur par défaut d'un type générique

Vous aurez remarqué dans l'implémentation de la liste du dessus que si nous essayons d'obtenir un élément du tableau à un indice qui n'existe pas, nous aurons une erreur. Ce comportement est une bonne chose, il est important de gérer les cas limites. En l'occurrence ici, on délègue au tableau la gestion du cas limite.

On pourrait envisager de gérer nous-mêmes ce cas limite en affichant un message et en renvoyant une valeur nulle. Seulement, pour un objet `Voiture`, qui est un type référence, il est tout à fait pertinent d'avoir `null`. Mais pour un `int`, qui est un type valeur, cela n'a pas de sens.

C'est là qu'intervient le mot-clé `default`.

Comme son nom l'indique, il renvoie la valeur par défaut du type. Pour un type référence, c'est `null`, pour un type valeur c'est `0`. Ce qui donnerait :

```
1 public T ObtenirElement(int indice)
2 {
3     if (indice < 0 || indice >= nbElements)
4     {
5         Console.WriteLine("L'indice n'est pas bon");
6         return default(T);
7     }
8     return tableau[indice];
9 }
```

Les interfaces génériques

Les interfaces peuvent aussi être génériques. D'ailleurs, ça me fait penser que plus haut, je vous ai indiqué qu'un code était moche et que j'en parlerai plus tard...

Il s'agissait du chapitre sur les interfaces où nous avons implémentés l'interface `IComparable`.

Nous souhaitons comparer des voitures entre elles et nous avons obtenu le code suivant :

```
1 public class Voiture : IComparable
2 {
3     public string Couleur { get; set; }
4     public string Marque { get; set; }
5     public int Vitesse { get; set; }
6
7     public int CompareTo(object obj)
8     {
9         Voiture voiture = (Voiture)obj;
10        return Vitesse.CompareTo(voiture.Vitesse);
11    }
12 }
```

Je souhaite pouvoir comparer des voitures entre elles, mais le framework .NET me fournit un `object` en paramètres de la méthode `CompareTo()`. Quelle idée ! Comme si je voulais pouvoir comparer des voitures avec des chats ou des chiens.

Cela me force en plus à faire un cast. Pourquoi il ne me passe pas directement une `Voiture` en paramètres ?

Vous le sentez venir et vous avez raison. Un `object` ! Berk, oserais-je dire ! Et sans parler des performances.

C'est là où les génériques vont pouvoir voler à notre secours. L'interface `IComparable` date de la première version du framework .NET. Le C# ne possédait pas encore les types génériques. Depuis leur apparition, il est possible d'implémenter la version générique de cette interface.

Pour cela, nous faisons suivre l'interface du type que nous souhaitons utiliser entre les chevrons. Cela donne :

```
1public class Voiture : IComparable<Voiture>
2{
3    public string Couleur { get; set; }
4    public string Marque { get; set; }
5    public int Vitesse { get; set; }
6
7    public int CompareTo(Voiture obj)
8    {
9        return Vitesse.CompareTo(obj.Vitesse);
10    }
11}
```

Nous devons toujours implémenter la méthode CompareTo() sauf que nous avons désormais un objet Voiture en paramètres, ce qui nous évite de le caster.

Les restrictions sur les types génériques

Une méthode ou une classe générique c'est bien, mais peut-être voulons-nous qu'elles ne fonctionnent pas avec tous les types.

Aussi, le C# permet de définir des restrictions sur les types génériques. Pour ce faire, on utilise le mot-clé where.

Il existe 6 types de restrictions :

Contrainte	Description
where T : struct	Le type générique doit être un type valeur
where T : class	Le type générique doit être un type référence
where T : new()	Le type générique doit posséder un constructeur par défaut
where T : IMonInterface	Le type générique doit implémenter l'interface IMonInterface
where T : MaClasse	Le type générique doit dériver de la classe MaClasse
where T1 : T2	Le type générique doit dériver du type générique T2

Par exemple, nous pouvons définir une restriction sur une méthode générique afin qu'elle n'accepte en type générique que des types qui implémentent une interface.

Soit l'interface suivante :

```
1public interface IVolant
2{
3    void DeplierLesAiles();
4    void Voler();
5}
```

Qui est implémentée par deux objets :

```
1public class Avion : IVolant
2{
3    public void DeplierLesAiles()
4    {
5        Console.WriteLine("Je déplie mes ailes mécaniques");
6    }
7
8    public void Voler()
9    {
```



```

10     Console.WriteLine("J'allume le moteur");
11 }
12}
13
14public class Oiseau : IVolant
15{
16     public void DeplierLesAiles()
17     {
18         Console.WriteLine("Je déplie mes ailes d'oiseau");
19     }
20
21     public void Voler()
22     {
23         Console.WriteLine("Je bats des ailes");
24     }
25}

```

Nous pouvons créer une méthode générique qui s'occupe d'instancier ces objets et d'appeler les méthodes de l'interface :

```

1public static T Creer<T>() where T : IVolant, new()
2{
3    T t = new T();
4    t.DeplierLesAiles();
5    t.Voler();
6    return t;
7}

```

Ici, la restriction se porte sur deux niveaux. Il faut dans un premier temps que le type générique implémente l'interface `IVolant` et possède également un constructeur, bref qu'il soit instanciable.

Nous pouvons donc utiliser cette méthode de cette façon :

```

1Oiseau oiseau = Creer<Oiseau>();
2Avion a380 = Creer<Avion>();

```

Nous appelons la méthode `Créer()` avec le type générique `Oiseau`, qui implémente bien `IVolant` et qui est aussi instanciable. Grâce à cela, nous pouvons utiliser l'opérateur `new` pour créer notre type générique, appeler les méthodes de l'interface et renvoyer l'instance.

Ce qui donne :

```

Je déplie mes ailes d'oiseau
Je bats des ailes
Je déplie mes ailes mécaniques
J'allume le moteur

```

Si nous tentons d'utiliser la méthode avec un type qui n'implémente pas l'interface `IVolant`, comme :

```

1Voiture v = Creer<Voiture>();

```

Nous aurons l'erreur de compilation suivante :

Le type '`MaPremiereApplication.Voiture`' ne peut pas être utilisé comme paramètre de type '`T`' dans le type ou la méthode générique '`MaPremiereApplication.Program.Creer<T>()`'. Il n'y a pas de conversion de référence implicite de '`MaPremiereApplication.Voiture`' en '`MaPremiereApplication.IVolant`'. Globalement, il nous dit que l'objet `Voiture` n'implémente pas `IVolant`.

Oui mais dans ce cas, plutôt que d'utiliser une méthode générique, pourquoi la méthode ne renvoie pas `IVolant` ?

C'est une judicieuse remarque. Mais qui nécessite quelques modifications de code. En effet, il faudrait indiquer quel type instancier.

Nous pourrions par exemple faire :

```
1public enum TypeDeVolant
2{
3    Oiseau,
4    Avion
5}
6
7public static IVolant Creer(TypeDeVolant type)
8{
9    IVolant volant;
10    switch (type)
11    {
12        case TypeDeVolant.Oiseau:
13            volant = new Oiseau();
14            break;
15        case TypeDeVolant.Avion:
16            volant = new Avion();
17            break;
18        default:
19            return null;
20    }
21    volant.DeplierLesAiles();
22    volant.Voler();
23    return volant;
24}
```

Et instancier nos objets de cette façon :

```
1Oiseau oiseau = (Oiseau)Creer(TypeDeVolant.Oiseau);
2Avion a380 = (Avion)Creer(TypeDeVolant.Avion);
```

Ce qui complique un peu les choses et rajoute des cast dont on pourrait volontiers se passer. De plus, si nous créons un nouvel objet qui implémente cette interface, il faudrait tout modifier.

Avouez qu'avec les types génériques, c'est quand même plus propre. 🤖

Nous pouvons bien sûr avoir des restrictions sur les types génériques d'une classe.

Pour le montrer, nous allons créer une classe dont l'objectif est d'avoir des types valeur qui pourraient ne pas avoir de valeur. Pour les types référence, il suffit d'utiliser le mot-clé `null`. Mais pour les types valeur comme les entiers, nous n'avons rien pour indiquer que ceux-ci n'ont pas de valeur.

Par exemple :

```
1public class TypeValeurNull<T> where T : struct
2{
3    private bool aUneValeur;
4    public bool AUneValeur
5    {
6        get { return aUneValeur; }
7    }
8
9    private T valeur;
10    public T Valeur
11    {
12        get
13        {
14            if (aUneValeur)
15                return valeur;
16            throw new InvalidOperationException();
17        }
18        set
```

```

19     {
20         aUneValeur = true;
21         valeur = value;
22     }
23 }
24}

```

Ici, nous utilisons une classe possédant un type générique qui sera un type valeur, grâce à la condition `where T : struct`. Cette classe encapsule le type générique pour indiquer avec un booléen si le type a une valeur ou pas.

Ne faites pas attention à la ligne :

```
1throw new InvalidOperationException();
```

qui permet juste de renvoyer une erreur, nous étudierons les exceptions un peu plus loin. Elle pourra s'utiliser ainsi :

```

1TypeValeurNull<int> entier = new TypeValeurNull<int>();
2if (!entier.AUneValeur)
3{
4    Console.WriteLine("l'entier n'a pas de valeur");
5}
6entier.Valeur = 5;
7if (entier.AUneValeur)
8{
9    Console.WriteLine("Valeur de l'entier : " + entier.Valeur);
10}

```

Et si nous souhaitons avoir pareil pour un autre type valeur, il n'y a rien à faire de plus :

```
1TypeValeurNull<double> valeur = new TypeValeurNull<double>();
```

C'est quand même super pratique comme classe !! Mais ne rêvons-pas, cette idée ne vient pas de moi. C'est en fait une fonctionnalité du framework .NET : les types nullable.

Les types nullable

En fait, la classe que nous avons vue au-dessus existe déjà dans le framework .NET, et en mieux ! Évidemment. Elle permet de faire exactement ce que j'ai décrit, c'est-à-dire permettre à un type valeur d'avoir une valeur nulle.

Elle est mieux faite dans la mesure où elle tire parti de certaines fonctionnalités du framework .NET qui en simplifie l'écriture. Il s'agit de la classe `Nullable<>`.

Aussi, nous pourrions créer un entier pouvant être `null` grâce au code suivant :

```

1Nullable<int> entier = null;
2if (!entier.HasValue)
3{
4    Console.WriteLine("l'entier n'a pas de valeur");
5}
6entier = 5;
7if (entier.HasValue)
8{
9    Console.WriteLine("Valeur de l'entier : " + entier.Value);
10}

```

Le principe est grosso modo le même sauf que nous pouvons utiliser le mot-clé `null` ou affecter directement la valeur à l'entier en utilisant l'opérateur d'affectation, sans passer par la propriété `Valeur`. Il peut aussi être comparé au mot-clé `null` ou être utilisé avec l'opérateur `+`, etc. Ceci est possible grâce à certaines fonctionnalités du C# que nous n'avons pas vues et qui sortent de l'étude de ce tutoriel.

Cette classe est tellement pratique que le compilateur a été optimisé pour simplifier son écriture. En effet, utiliser `Nullable<>` est un peu long pour nous autres informaticiens qui sommes des paresseux. 😊
Aussi, l'écriture :

```
1Nullable<int> entier = null;
```

peut se simplifier en :

```
1int? entier = null;
```

C'est le point d'interrogation qui remplace la déclaration de la classe `Nullable<>`.

En résumé

- Avec les génériques, vous pouvez créer des méthodes ou des classes qui sont indépendantes d'un type. On les appellera des méthodes génériques et des types génériques.
- On utilise les chevrons `<>` pour indiquer le type d'une classe ou d'une méthode générique.
- Les interfaces peuvent aussi être génériques, comme l'interface `IEnumerable<>`.
- Les types nullable constituent un exemple d'utilisation très pratique des classes génériques.